

EXPERIMENT – 07

Experiment No.	07
Date	_____
Title	Method Overriding – Bank Interest Rate and Dynamic Polymorphism

1. Objective / Aim

To implement bank interest-rate functionality using method overriding and demonstrate dynamic polymorphism in Java.

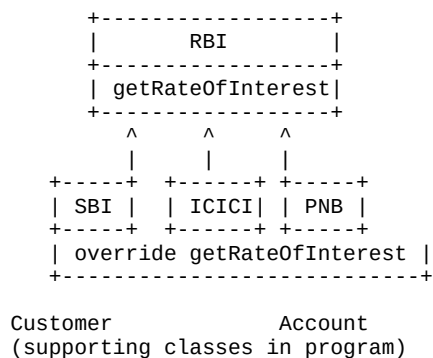
2. System Requirements

Hardware	Computer/Laptop with keyboard and monitor
Software	JDK (Java Development Kit), Command Prompt or any Java IDE

3. Algorithm / Logic Description

1. Define RBI as the base class with a 4% interest rate.
2. Create SBI, ICICI, and PNB subclasses and override `getRateOfInterest()` in each class.
3. Read the bank name from the user.
4. Store the selected bank object in an RBI reference.
5. Call `getRateOfInterest()`; Java selects the overridden method according to the actual object at runtime.
6. Display the bank name and interest rate. If the name is invalid, display an error message.

4. UML Class Diagram



5. Source Code

```
import java.util.Scanner;

class RBI {
    double getRateOfInterest() {
        return 4.0;
    }
}

class SBI extends RBI {
    @Override
    double getRateOfInterest() {
        return 7.0;
    }
}

class ICICI extends RBI {
    @Override
    double getRateOfInterest() {
        return 6.5;
    }
}

class PNB extends RBI {
```

```

        @Override
        double getRateOfInterest() {
            return 6.0;
        }
    }

    class Account {
        String accountHolder;
        double balance;

        Account(String accountHolder, double balance) {
            this.accountHolder = accountHolder;
            this.balance = balance;
        }

        void displayAccount() {
            System.out.println("Customer Name: " + accountHolder);
            System.out.println("Balance      : " + balance);
        }
    }

    class Customer {
        String name;

        Customer(String name) {
            this.name = name;
        }

        void displayCustomer() {
            System.out.println("Customer Name: " + name);
        }
    }

    public class BankInterestDemo {
        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter the Bank name to find the rate of Interest: ");
            String bankName = sc.nextLine().trim();

            // RBI reference demonstrates dynamic polymorphism.
            RBI bank;

            if (bankName.equalsIgnoreCase("RBI")) {
                bank = new RBI();
            } else if (bankName.equalsIgnoreCase("SBI")) {
                bank = new SBI();
            } else if (bankName.equalsIgnoreCase("ICICI")) {
                bank = new ICICI();
            } else if (bankName.equalsIgnoreCase("PNB")) {
                bank = new PNB();
            } else {
                System.out.println("Invalid bank name. Please enter RBI, SBI, ICICI, or PNB.");
                sc.close();
                return;
            }

            System.out.println(bankName.toUpperCase()
                + " rate of interest is: "
                + bank.getRateOfInterest() + "%");

            sc.close();
        }
    }
}

```

6. Compilation & Execution Commands

```

javac BankInterestDemo.java
java BankInterestDemo

```

7. Output Screen

Test Case 1 – RBI

```

Enter the Bank name to find the rate of Interest: RBI
RBI rate of interest is: 4.0%

```

Test Case 2 – SBI

```

Enter the Bank name to find the rate of Interest: SBI
SBI rate of interest is: 7.0%

```

Test Case 3 – ICICI

```

Enter the Bank name to find the rate of Interest: ICICI
ICICI rate of interest is: 6.5%

```

Test Case 4 – PNB

Enter the Bank name to find the rate of Interest: PNB
PNB rate of interest is: 6.0%

Test Case 5 – Invalid Bank

Enter the Bank name to find the rate of Interest: ABC
Invalid bank name. Please enter RBI, SBI, ICICI, or PNB.

8. Result

Thus, the Java program demonstrating method overriding and dynamic polymorphism for bank interest rates was implemented and executed successfully. The RBI base-class reference called the appropriate overridden method for RBI, SBI, ICICI, or PNB at runtime.